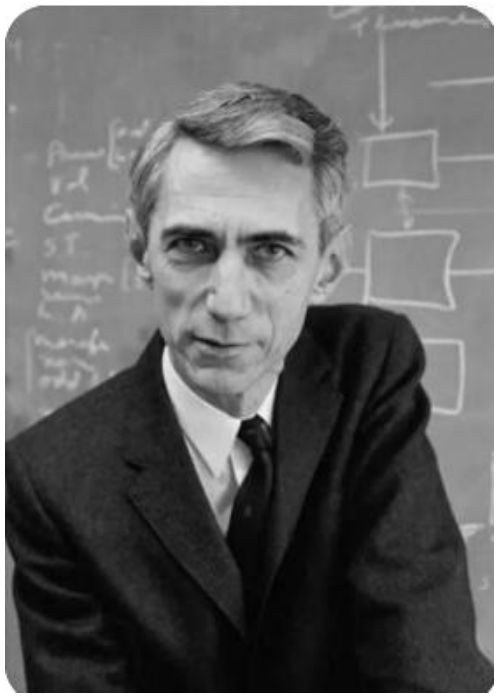
INFORMATION THEORY & CODING



Lecture 6

Review Summary

- **Classes of codes**

  Prefix codes $\Rightarrow$ Uniquely decodable codes $\Rightarrow$ Nonsingular codes

- **Kraft inequality**

  Prefix codes $\Leftrightarrow \sum D^{-\ell_i} \leq 1$.

- **Entropy bound on data compression**

  $L \stackrel{\Delta}{=} \sum p_i \ell_i \geq H_D(X)$.

Review Summary

- **Shannon code**

$$\ell_i = \lceil \log_D \frac{1}{p_i} \rceil$$

$$H_D(X) \leq L < H_D(X) + 1.$$

Kraft Inequaltiy for Uniquely Decodable Codes

- **Theorem 5.5.1** (*McMillan*) The codeword lengths of any uniquely decodable D-ary code must satisfy the Kraft inequality

$$\sum D^{-\ell_i} \leq 1.$$

Conversely, given a set of codeword lengths that satisfy this inequality, it is possible to construct a uniquely decodable code with these codeword lengths.

Kraft Inequaltiy for Uniquely Decodable Codes

- **Theorem 5.5.1** (*McMillan*) The codeword lengths of any uniquely decodable D-ary code must satisfy the Kraft inequality

$$\sum D^{-\ell_i} \leq 1.$$

Conversely, given a set of codeword lengths that satisfy this inequality, it is possible to construct a uniquely decodable code with these codeword lengths.

Proof. Consider C^k, the kth extension of the code by k repetitions. Let the codeword lengths of the symbols $x \in \mathcal{X}$ be $\ell(x)$. For the kth extension code, we have

$$\ell(x_1, x_2, \ldots, x_k) = \sum_{i=1}^{k} \ell(x_i).$$

Kraft Inequaltiy for Uniquely Decodable Codes

- **Theorem 5.5.1** (*McMillan*) The codeword lengths of any uniquely decodable D-ary code must satisfy the Kraft inequality

$$\sum D^{-\ell_i} \leq 1.$$

Proof. (cont.) Consider

$$\begin{aligned}
\left(\sum_{x\in\mathcal{X}} D^{-\ell(x)}\right)^k &= \sum_{x_1\in\mathcal{X}}\sum_{x_2\in\mathcal{X}}\cdots\sum_{x_k\in\mathcal{X}} D^{-\ell(x_1)}D^{-\ell(x_2)}\cdots D^{-\ell(x_k)} \\
&= \sum_{x_1,x_2,\cdots x_k\in\mathcal{X}^k} D^{-\ell(x_1)}D^{-\ell(x_2)}\cdots D^{-\ell(x_k)} \\
&= \sum_{x^k\in\mathcal{X}^k} D^{-\ell(x^k)},
\end{aligned}$$

Kraft Inequaltiy for Uniquely Decodable Codes

- **Theorem 5.5.1** (*McMillan*) The codeword lengths of any uniquely decodable D-ary code must satisfy the Kraft inequality

$$\sum D^{-\ell_i} \leq 1.$$

Proof. (cont.) Let $\ell_{\max}$ be the maximum codeword length and $a(m)$ is the number of source sequences x^k mapping into codewords of length m. Unique decodability implies that $a(m) \leq D^m$. We have

$$\begin{aligned}
\left(\sum_{x \in \mathcal{X}} D^{-\ell(x)} \right)^k &= \sum_{m=1}^{k\ell_{\max}} a(m) D^{-m} \\
&\leq \sum_{m=1}^{k\ell_{\max}} D^m D^{-m} \\
&= k\ell_{\max}.
\end{aligned}$$

Kraft Inequaltiy for Uniquely Decodable Codes

- **Theorem 5.5.1** (*McMillan*) The codeword lengths of any uniquely decodable D-ary code must satisfy the Kraft inequality

$$\sum D^{-\ell_i} \leq 1.$$

Proof. (cont.)

$$\left(\sum_{x \in \mathcal{X}} D^{-\ell(x)} \right)^k \leq k\ell_{\max}.$$

Hence,

$$\sum_j D^{-\ell_j} \leq (k\ell_{\max})^{1/k}$$

holds for all k. Since the RHS $\to 1$ as $k \to \infty$, we prove the Kraft inequality.
For the converse part, we can construct a prefix code as in **Theorem 5.2.1**, which is also uniquely decodable.

Huffman Codes

- **Problem** Given source symbols and their probabilities of occurence, how to design an optimal source code (prefix code and the shortest on average)?

Huffman Codes

- **Problem** Given source symbols and their probabilities of occurence, how to design an optimal source code (prefix code and the shortest on average)?

- **Huffman Codes**

  Step 1. Merge the D smallest symbol probabilites;
  Step 2. Assign the D corresponding symbols with $0, 1, \ldots, D-1$, then go back to Step 1.

  Repeat the above process until D probabilities are merged into probability 1.

David A. Huffman, "A method of the construction of minimum redundancy codes," Proc. IRE, vol. 40, pp. 1098–1101, 1952.

Huffman Codes: A few examples

- **Example 1**

x	p(x)
1	0.25
2	0.25
3	0.2
4	0.15
5	0.15

Huffman Codes: A few examples

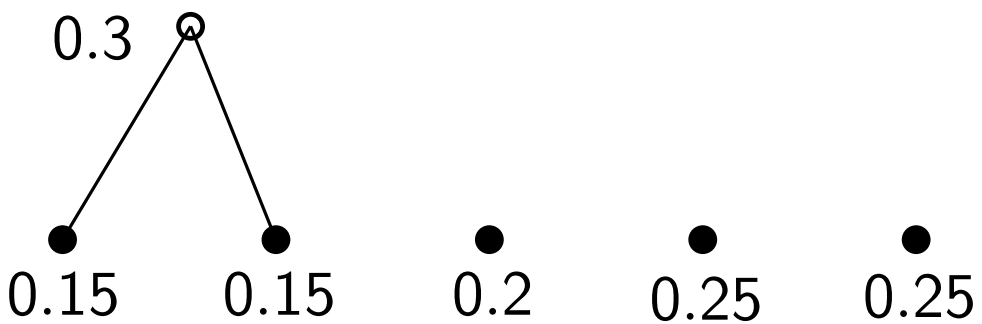
- **Example 1**

x	p(x)
1	0.25
2	0.25
3	0.2
4	0.15
5	0.15

Huffman Codes: A few examples

- **Example 1**

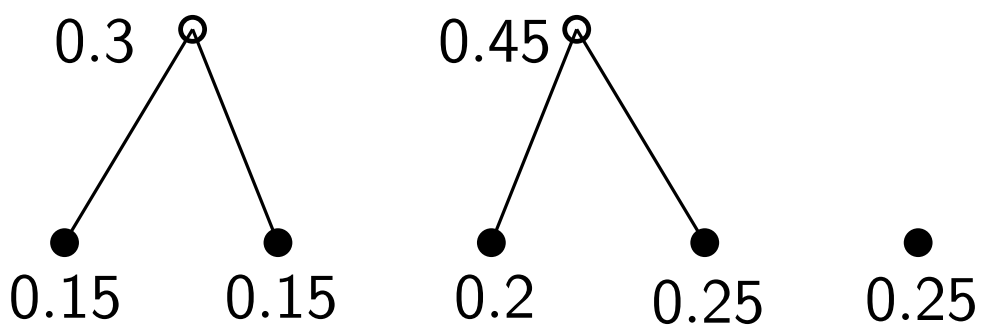
x	p(x)
1	0.25
2	0.25
3	0.2
4	0.15
5	0.15



Huffman Codes: A few examples

- **Example 1**

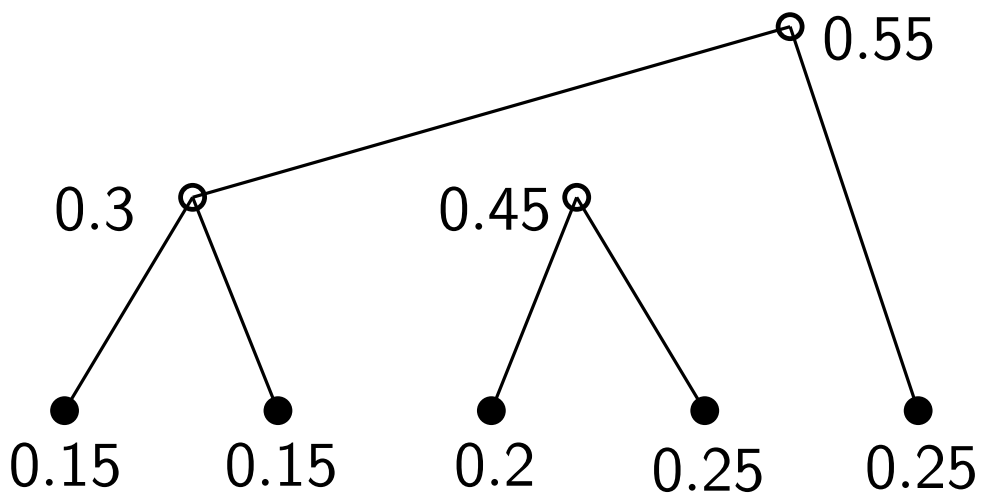
x	p(x)
1	0.25
2	0.25
3	0.2
4	0.15
5	0.15



Huffman Codes: A few examples

- **Example 1**

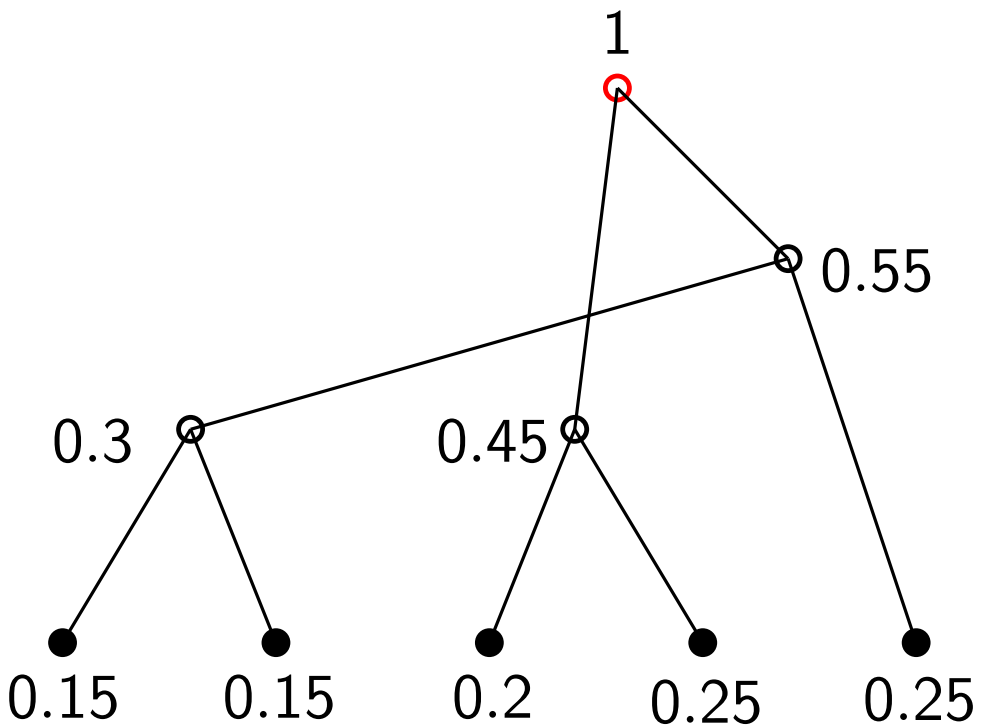
x	p(x)
1	0.25
2	0.25
3	0.2
4	0.15
5	0.15



Huffman Codes: A few examples

- **Example 1**

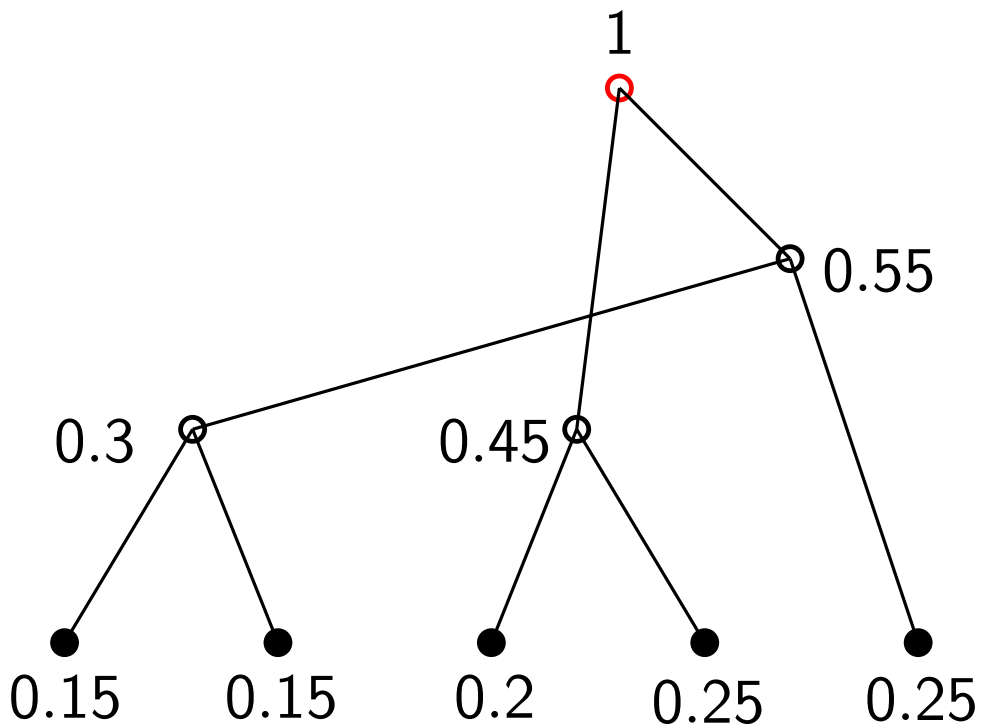
x	p(x)
1	0.25
2	0.25
3	0.2
4	0.15
5	0.15



Huffman Codes: A few examples

- **Example 1**

x	p(x)
1	0.25
2	0.25
3	0.2
4	0.15
5	0.15

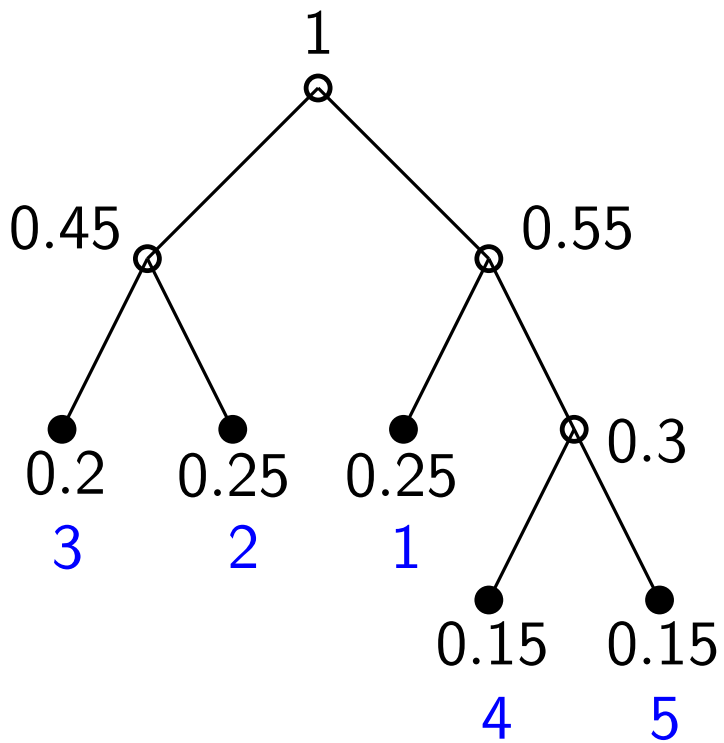


Reconstruct the tree

Huffman Codes: A few examples

- **Example 1**

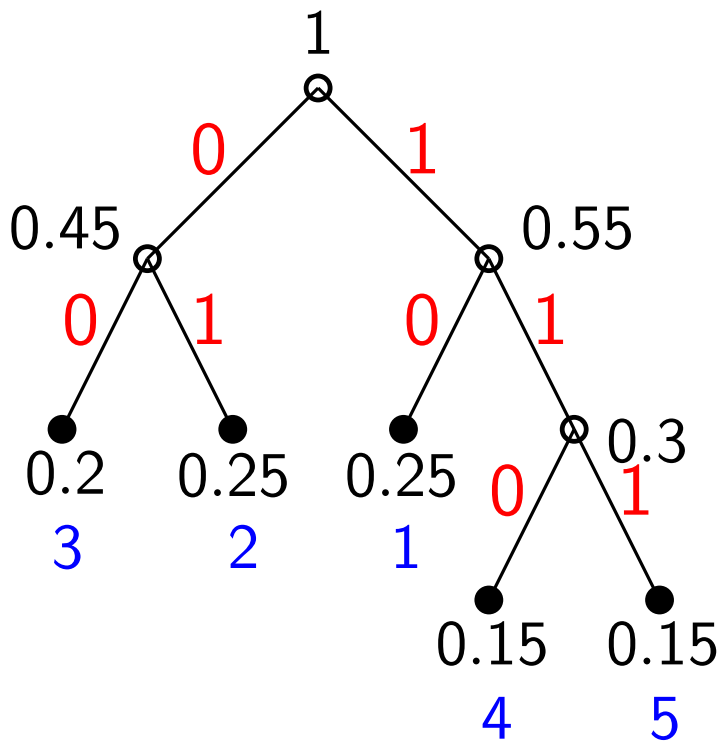
x	p(x)
1	0.25
2	0.25
3	0.2
4	0.15
5	0.15



Huffman Codes: A few examples

- **Example 1**

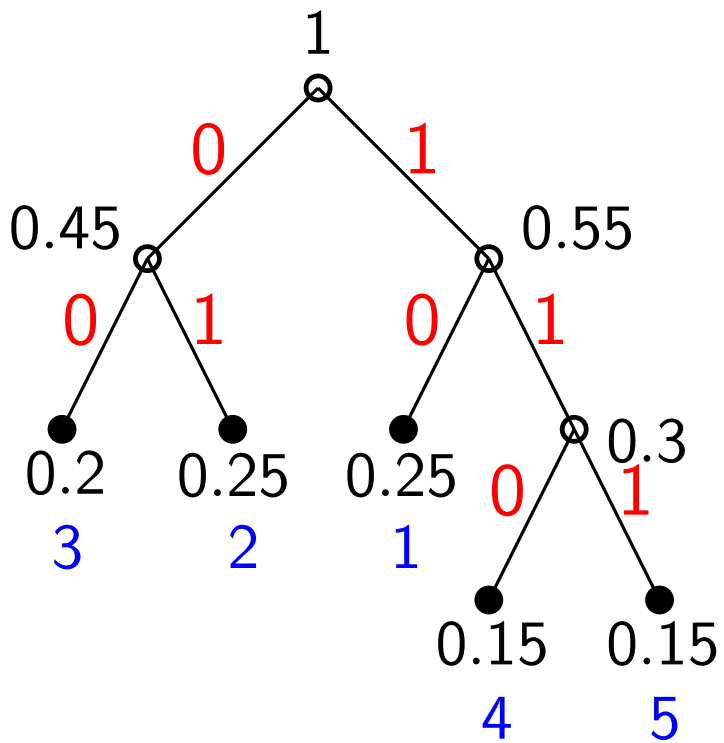
x	p(x)
1	0.25
2	0.25
3	0.2
4	0.15
5	0.15



Huffman Codes: A few examples

- **Example 1**

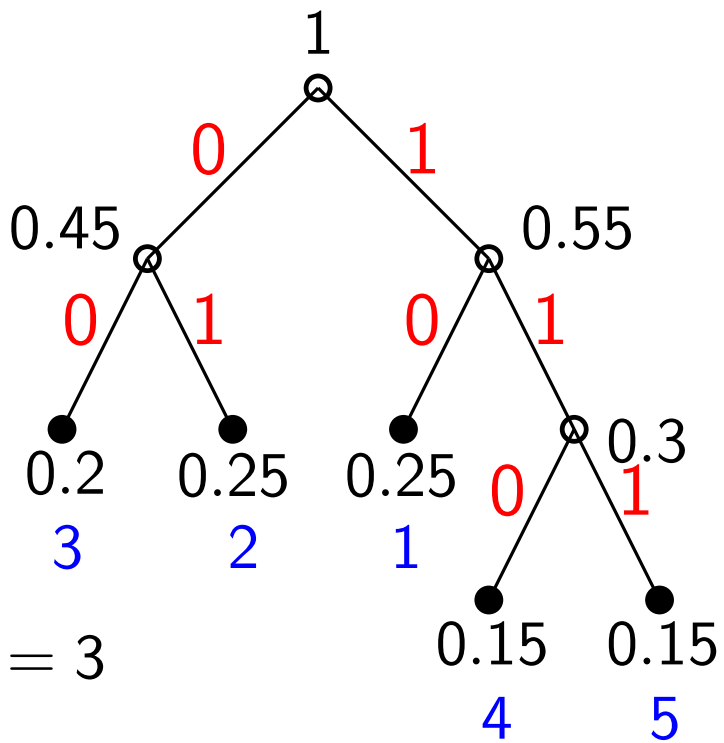
x	p(x)	C(x)
1	0.25	10
2	0.25	01
3	0.2	00
4	0.15	110
5	0.15	111



Huffman Codes: A few examples

- **Example 1**

x	p(x)	C(x)
1	0.25	10
2	0.25	01
3	0.2	00
4	0.15	110
5	0.15	111



Validations:

$\ell(1) = \ell(2) = \ell(3) = 2,\ \ell(4) = \ell(5) = 3$

$L = \sum \ell(x)p(x) = 2.3$ bits

$H_2(X) = -\sum p(x) \log_2 p(x) = 2.3$ bits

$$L \geq H_2(X)$$

Huffman Codes: A few examples

- **Example 2** ($D \geq 3$)

x	p(x)
1	0.25
2	0.25
3	0.2
4	0.1
5	0.1
6	0.1

Huffman Codes: A few examples

- **Example 2** ($D \geq 3$)

x	p(x)
1	0.25
2	0.25
3	0.2
4	0.1
5	0.1
6	0.1

At one time, we merge D symbols, and at each stage of the reduction, the number of symbols is reduced by $D-1$. We want the total # of symbols to be $1+k(D-1)$. If not, we add dummy symbols with probability 0.

Huffman Codes: A few examples

- **Example 2** ($D \geq 3$)

x	p(x)
1	0.25
2	0.25
3	0.2
4	0.1
5	0.1
6	0.1
Dummy	0

At one time, we merge D symbols, and at each stage of the reduction, the number of symbols is reduced by $D-1$. We want the total # of symbols to be $1+k(D-1)$. If not, we add dummy symbols with probability 0.

Huffman Codes: A few examples

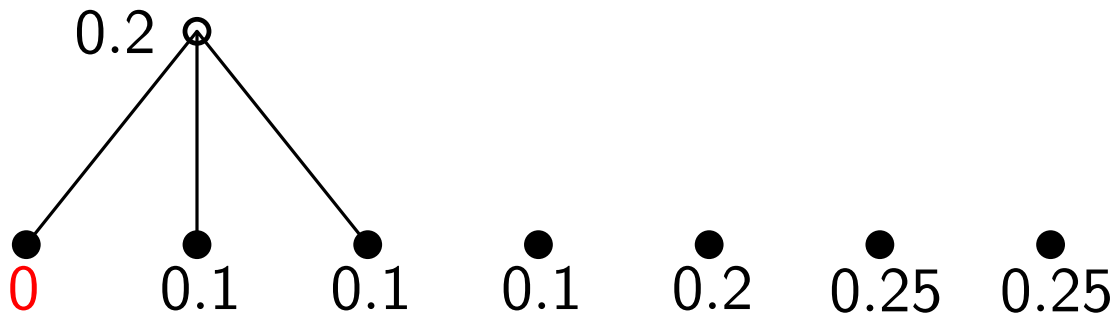
- **Example 2** ($D \geq 3$)

x	p(x)
1	0.25
2	0.25
3	0.2
4	0.1
5	0.1
6	0.1
Dummy	0

Huffman Codes: A few examples

- **Example 2** ($D \geq 3$)

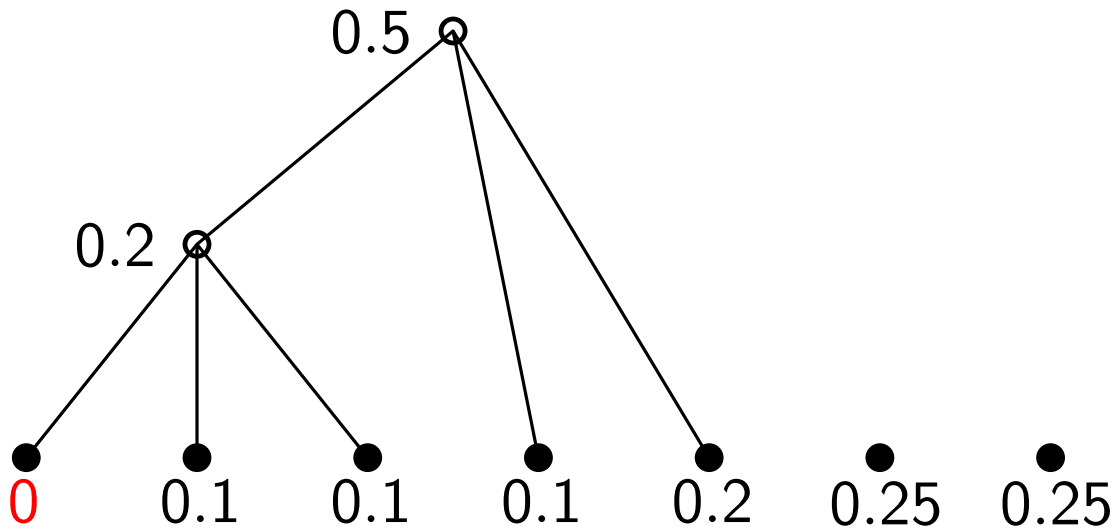
x	p(x)
1	0.25
2	0.25
3	0.2
4	0.1
5	0.1
6	0.1
Dummy	0



Huffman Codes: A few examples

- **Example 2** ($D \geq 3$)

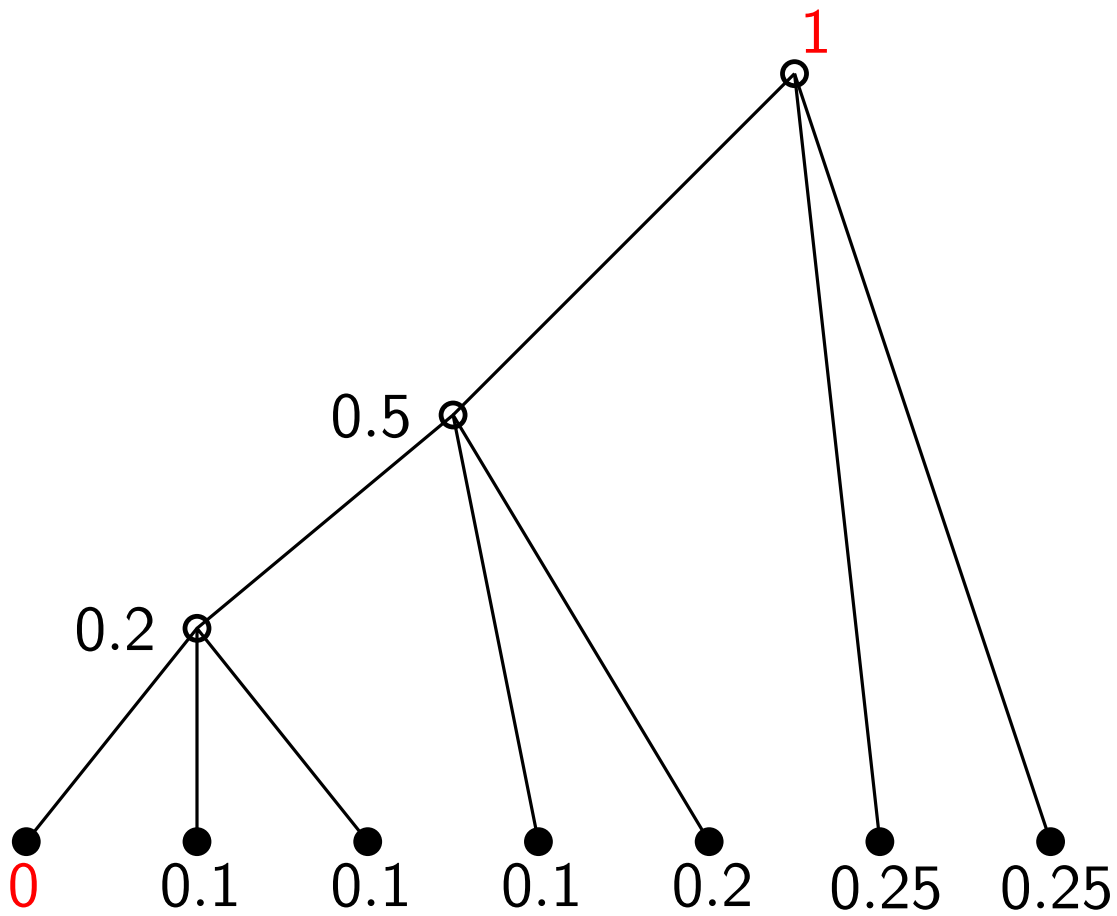
x	p(x)
1	0.25
2	0.25
3	0.2
4	0.1
5	0.1
6	0.1
Dummy	0



Huffman Codes: A few examples

- **Example 2** ($D \geq 3$)

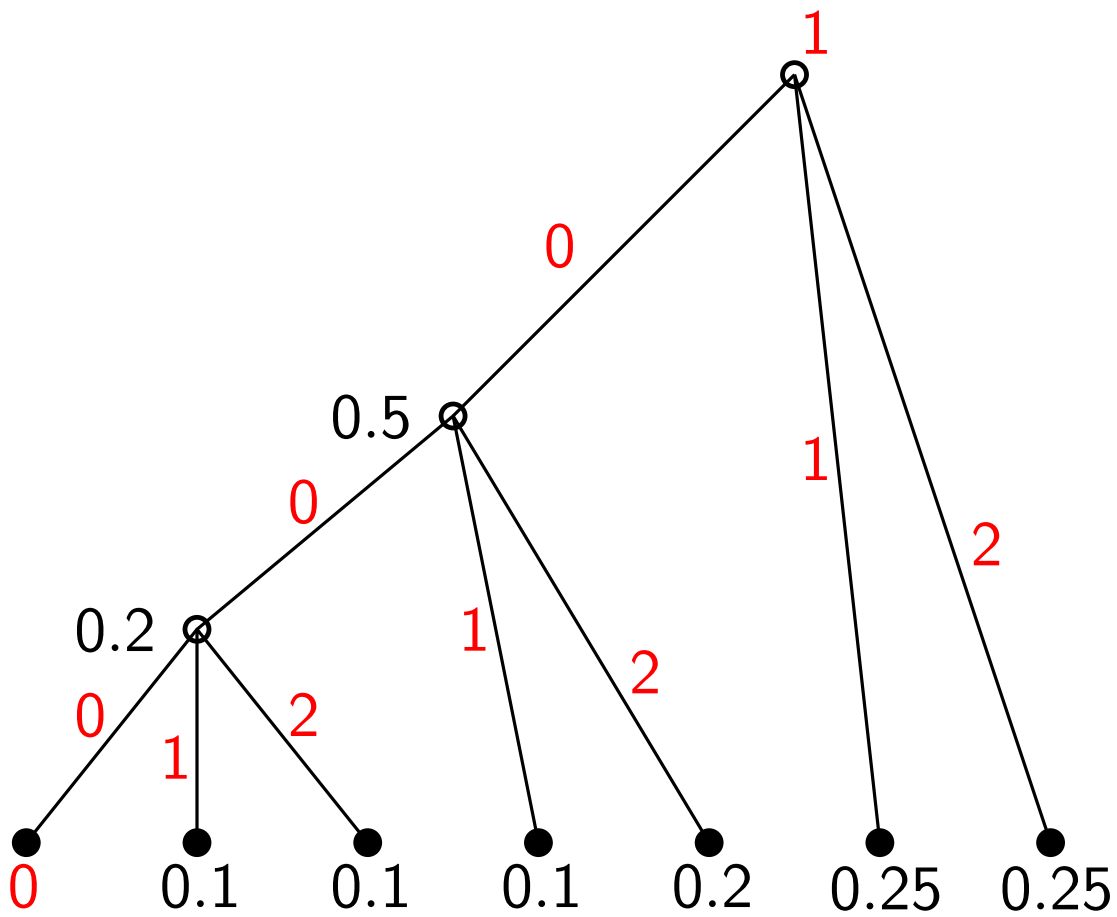
x	p(x)
1	0.25
2	0.25
3	0.2
4	0.1
5	0.1
6	0.1
Dummy	0



Huffman Codes: A few examples

- **Example 2** ($D \geq 3$)

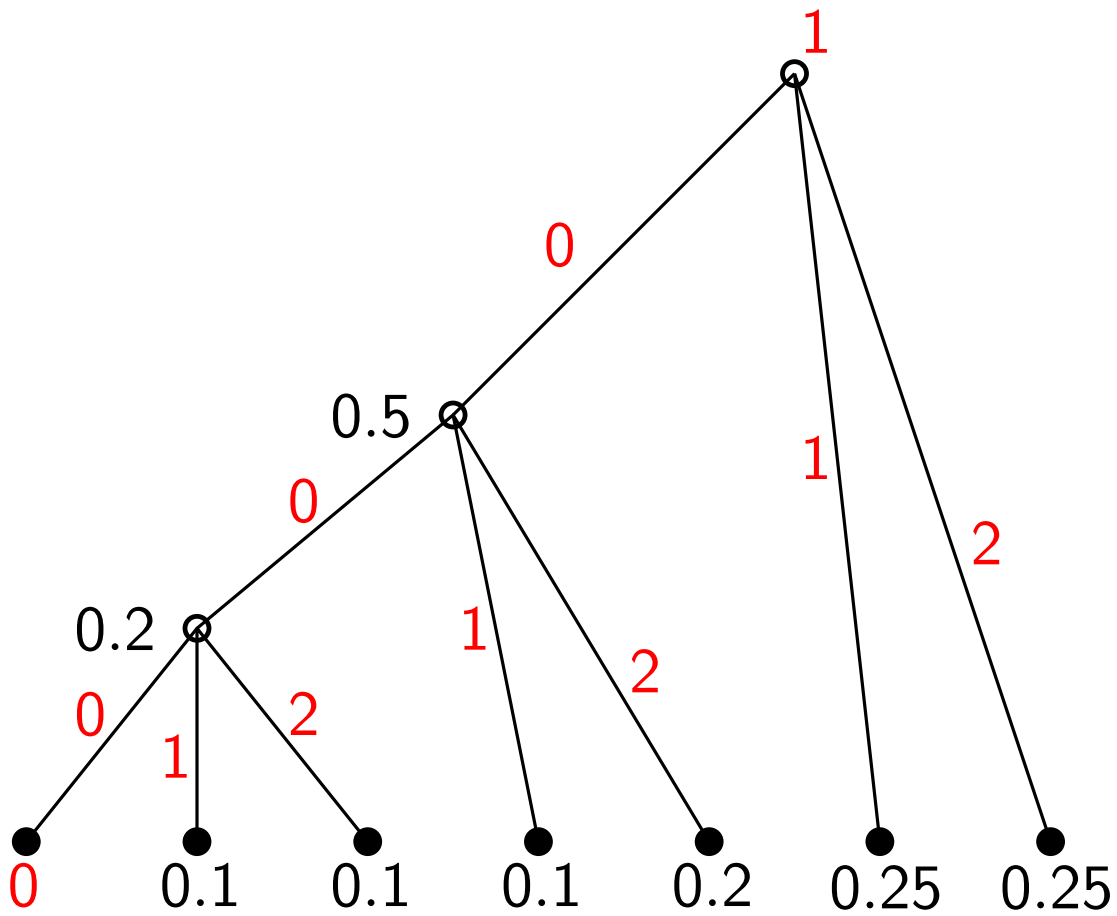
x	p(x)
1	0.25
2	0.25
3	0.2
4	0.1
5	0.1
6	0.1
Dummy	0



Huffman Codes: A few examples

- **Example 2** ($D \geq 3$)

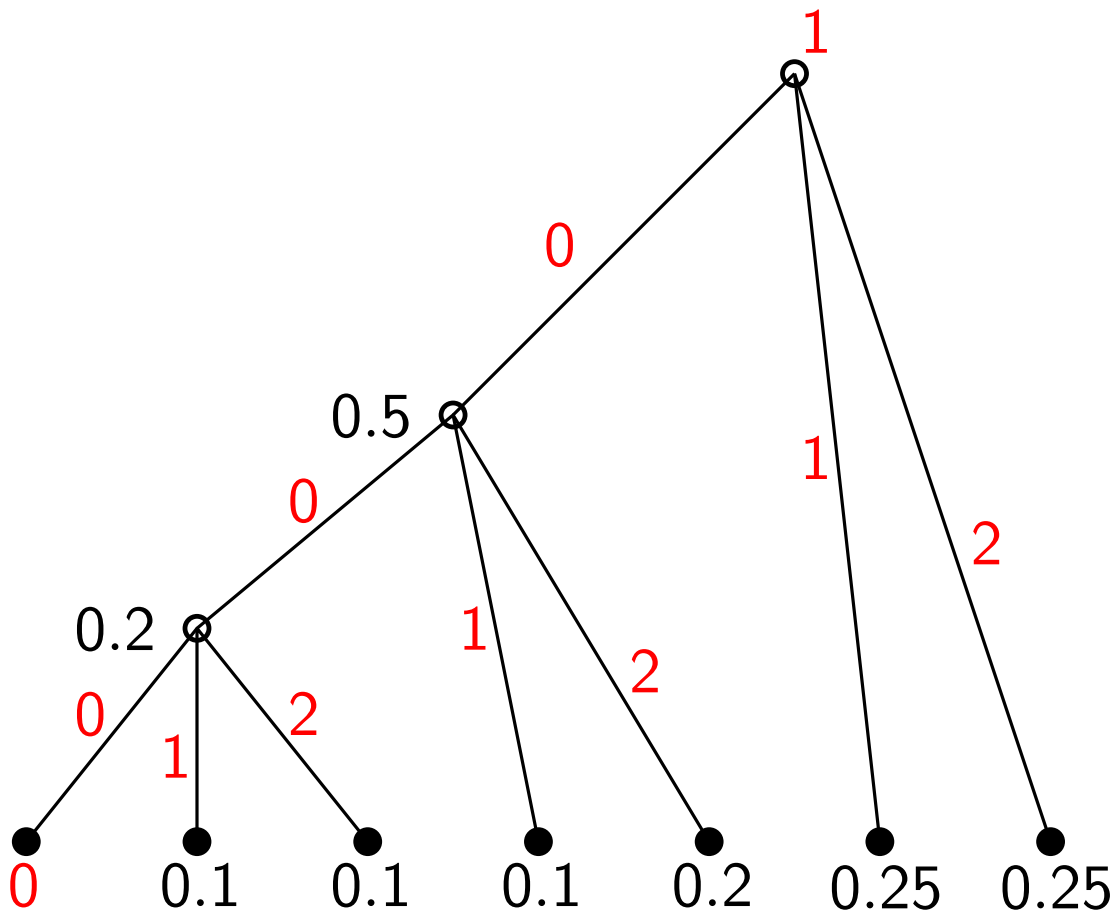
x	p(x)	C(x)
1	0.25	1
2	0.25	2
3	0.2	02
4	0.1	01
5	0.1	002
6	0.1	001
Dummy	0	000



Huffman Codes: A few examples

- **Example 2** ($D \geq 3$)

x	p(x)	C(x)
1	0.25	1
2	0.25	2
3	0.2	02
4	0.1	01
5	0.1	002
6	0.1	001
Dummy	0	000



Validations:

$L = \sum \ell(x)p(x) = 1.7$ ternary digits

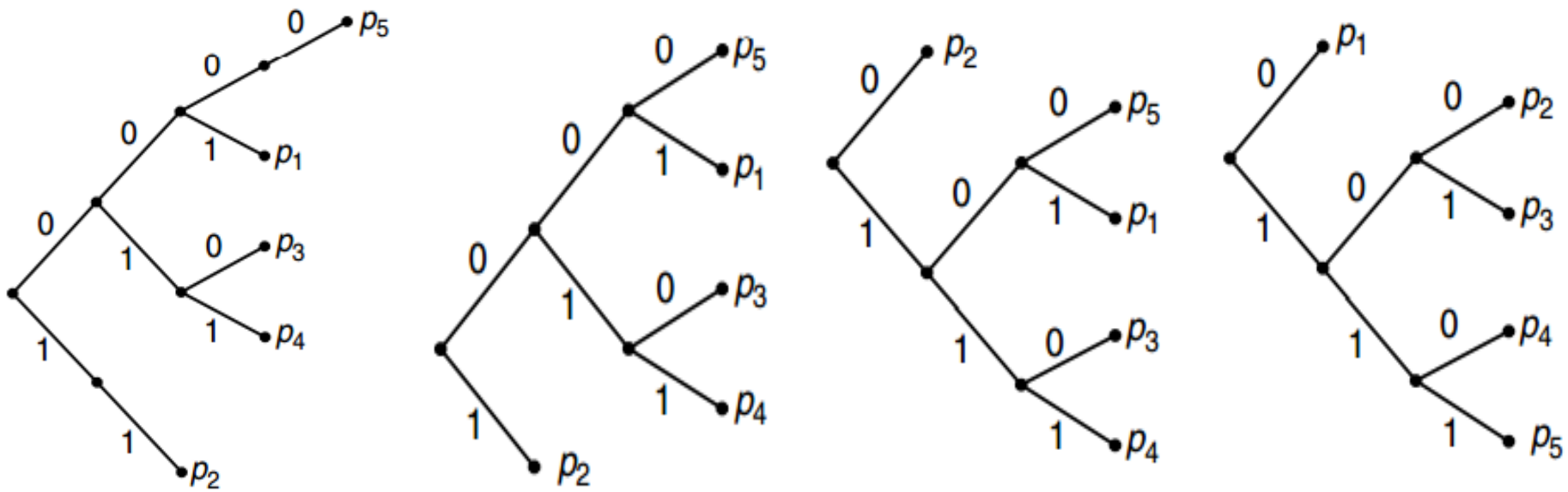
$H_3(X) = -\sum p(x) \log_3 p(x) \approx 1.55$ ternary digits

Optimality of Huffman Codes

- There are many optimal codes: inverting all the bits or exchanging two codewords of the same length will give another optimal code. W.l.o.g., assume that $p_1 \geq p_2 \geq \cdots \geq p_m$. Recall that a code is *optimal* if $\sum p_i \ell_i$ is minimal.

Optimality of Huffman Codes

- There are many optimal codes: inverting all the bits or exchanging two codewords of the same length will give another optimal code. W.l.o.g., assume that $p_1 \geq p_2 \geq \cdots \geq p_m$. Recall that a code is *optimal* if $\sum p_i \ell_i$ is minimal.



Optimality of Huffman Codes

- **Lemma 5.8.1** For any distribution, there exists an optimal prefix code (*with minimum exptected length*) that satisfies the following properties:
  1. If $p_j > p_k$, then $\ell_j \leq \ell_k$.
  2. The two longest codewords have the same length.
  3. Two of the longest codewords differ only in the last bit and correspond to the two least likely symbols.

Optimality of Huffman Codes

- 1. If $p_j > p_k$, then $\ell_j \leq \ell_k$.

  Proof. Suppose that C_m is an optimal code. Consider C'_m, with the codewords j and k of C_m interchanged. Then

$$\begin{aligned} \underbrace{L(C'_m) - L(C_m)}_{\geq 0} &= \sum p_i \ell'_i - \sum p_i \ell_i \\ &= p_j \ell_k + p_k \ell_j - p_j \ell_j - p_k \ell_k \\ &= \underbrace{(p_j - p_k)}_{>0}(\ell_k - \ell_j). \end{aligned}$$

  Thus, we must have $\ell_k \geq \ell_j$.

Optimality of Huffman Codes

- 2. The two longest codewords have the same length.

  Proof. If the two longest codewords are NOT of the same length, one can delete the last bit of the longer one, preserving the prefix property and achieving lower expected codeword length, contradiction! By property 1, the longest codewords must belong to the least probable source symbols.

Optimality of Huffman Codes

- 3. Two of the longest codewords differ only in the last bit and correspond to the two least likely symbols.

Proof. If there is a maximal-length codeword without a sibling（兄弟姐妹）, we can delete the last bit of the codeword and still preserve the prefix property. This reduces the average codeword length and contradicts the optimality of the code. Hence, every maximum-length codeword in any optimal code has a sibling. Now we can exchange the longest codewords s.t. the two lowest-probability source symbols are associated with two siblings on the tree, without changing the expected length.

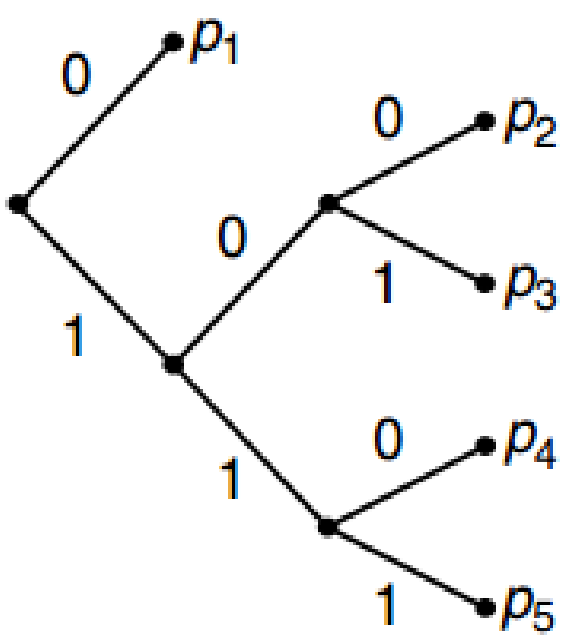
Optimality of Huffman Codes

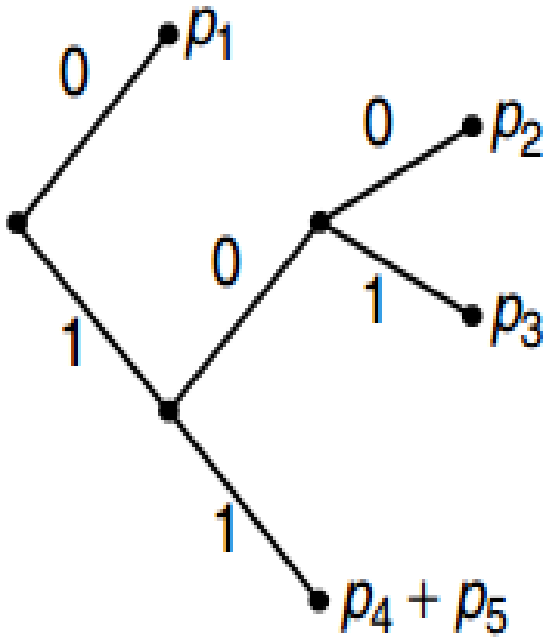
- **Lemma 5.8.1** For any distribution, there exists an optimal prefix code (*with minimum exptected length*) that satisfies the following properties:
  1. If $p_j > p_k$, then $\ell_j \leq \ell_k$.
  2. The two longest codewords have the same length.
  3. Two of the longest codewords differ only in the last bit and correspond to the two least likely symbols.

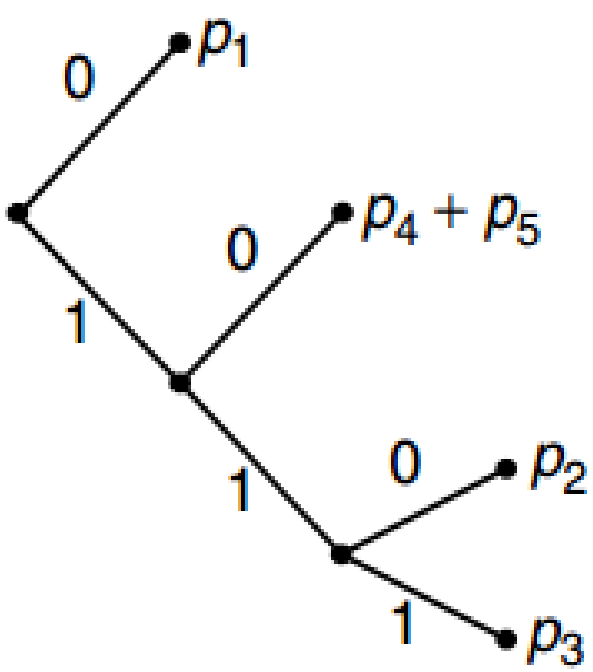
$\Rightarrow$ If $p_1 \geq p_2 \geq \cdots p_m$, then there exists an optimal code with $\ell_1 \leq \ell_2 \leq \cdots \ell_{m-1} = \ell_m$, and codewords $C(x_{m-1})$ and $C(x_m)$ differ only in the last bit. (*canonical codes*)（典则码）

Optimality of Huffman Codes

- We prove the *optimality* of Huffman codes by induction. （Huffman reduction 赫夫曼合并）







Optimality of Huffman Codes

- **Proof.** For $\mathbf{p} = (p_1, p_2, \ldots, p_m)$ with $p_1 \geq p_2 \geq \cdots \geq p_m$, we define the Huffman reduction $\mathbf{p}' = (p_1, p_2, \ldots, p_{m-1} + p_m)$ over an alphabet size of $m - 1$. Let $C^*_{m-1}(\mathbf{p}')$ be an optimal code for $\mathbf{p}'$, and let $C^*_m(\mathbf{p})$ be the canonical optimal code for $\mathbf{p}$.

Optimality of Huffman Codes

- **Proof.** For $\mathbf{p} = (p_1, p_2, \ldots, p_m)$ with $p_1 \geq p_2 \geq \cdots \geq p_m$, we define the Huffman reduction $\mathbf{p}' = (p_1, p_2, \ldots, p_{m-1} + p_m)$ over an alphabet size of $m - 1$. Let $C^*_{m-1}(\mathbf{p}')$ be an optimal code for $\mathbf{p}'$, and let $C^*_m(\mathbf{p})$ be the canonical optimal code for $\mathbf{p}$.

  Key idea.

  expand $C^*_{m-1}(\mathbf{p}')$ to $C_m(\mathbf{p})$
  condense $C^*_m(\mathbf{p})$ to $C_{m-1}(\mathbf{p}')$

Optimality of Huffman Codes

- **Proof.** For $\mathbf{p} = (p_1, p_2, \ldots, p_m)$ with $p_1 \geq p_2 \geq \cdots \geq p_m$, we define the Huffman reduction $\mathbf{p}' = (p_1, p_2, \ldots, p_{m-1} + p_m)$ over an alphabet size of $m - 1$. Let $C^*_{m-1}(\mathbf{p}')$ be an optimal code for $\mathbf{p}'$, and let $C^*_m(\mathbf{p})$ be the canonical optimal code for $\mathbf{p}$.

	$C^*_{m-1}(\mathbf{p}')$		$C_m(\mathbf{p})$	
p_1	w'_1	l'_1	$w_1 = w'_1$	$l_1 = l'_1$
p_2	w'_2	l'_2	$w_2 = w'_2$	$l_2 = l'_2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
p_{m-2}	w'_{m-2}	l'_{m-2}	$w_{m-2} = w'_{m-2}$	$l_{m-2} = l'_{m-2}$
$p_{m-1} + p_m$	w'_{m-1}	l'_{m-1}	$w_{m-1} = w'_{m-1}0$	$l_{m-1} = l'_{m-1} + 1$
			$w_m = w'_{m-1}1$	$l_m = l'_{m-1} + 1$

Optimality of Huffman Codes

- **Proof.** For $\mathbf{p} = (p_1, p_2, \ldots, p_m)$ with $p_1 \geq p_2 \geq \cdots \geq p_m$, we define the Huffman reduction $\mathbf{p}' = (p_1, p_2, \ldots, p_{m-1} + p_m)$ over an alphabet size of $m - 1$. Let $C^*_{m-1}(\mathbf{p}')$ be an optimal code for $\mathbf{p}'$, and let $C^*_m(\mathbf{p})$ be the canonical optimal code for $\mathbf{p}$.

  expand $C^*_{m-1}(\mathbf{p}')$ to $C_m(\mathbf{p})$

  $$L(\mathbf{p}) = L^*(\mathbf{p}') + p_{m-1} + p_m \quad \text{(PP. 72)}$$

  condense $C^*_m(\mathbf{p})$ to $C_{m-1}(\mathbf{p}')$

  $$L^*(\mathbf{p}) = L(\mathbf{p}') + p_{m-1} + p_m$$

Optimality of Huffman Codes

- **Proof.** For $\mathbf{p} = (p_1, p_2, \ldots, p_m)$ with $p_1 \geq p_2 \geq \cdots \geq p_m$, we define the Huffman reduction $\mathbf{p}' = (p_1, p_2, \ldots, p_{m-1} + p_m)$ over an alphabet size of $m - 1$. Let $C^*_{m-1}(\mathbf{p}')$ be an optimal code for $\mathbf{p}'$, and let $C^*_m(\mathbf{p})$ be the canonical optimal code for $\mathbf{p}$.

$$L(\mathbf{p}) = L^*(\mathbf{p}') + p_{m-1} + p_m$$

$$L^*(\mathbf{p}) = L(\mathbf{p}') + p_{m-1} + p_m$$

$$\underbrace{(L(\mathbf{p}') - L^*(\mathbf{p}'))}_{\geq 0} + \underbrace{(L(\mathbf{p}) - L^*(\mathbf{p}))}_{\geq 0} = 0$$

Optimality of Huffman Codes

- **Proof.** For $\mathbf{p} = (p_1, p_2, \ldots, p_m)$ with $p_1 \geq p_2 \geq \cdots \geq p_m$, we define the Huffman reduction $\mathbf{p}' = (p_1, p_2, \ldots, p_{m-1} + p_m)$ over an alphabet size of $m - 1$. Let $C^*_{m-1}(\mathbf{p}')$ be an optimal code for $\mathbf{p}'$, and let $C^*_m(\mathbf{p})$ be the canonical optimal code for $\mathbf{p}$.

  Thus, $L(\mathbf{p}) = L^*(\mathbf{p})$. Minimizing the expected length $L(C_m)$ is equivalent to minimizing $L(C_{m-1})$. The problem is reduced to one with $m - 1$ symbols and probability masses $(p_1, p_2, \ldots, p_{m-1} + p_m)$. Proceeding this way, we finally reduce the problem to two symbols, in which case the optimal code is obvious.

Optimality of Huffman Codes

- **Theorem 5.8.1.** Huffman coding is *optimal*, that is, if C^* is a Huffman code and C' is any other uniquely decodable code, $L(C^*) \leq L(C')$.

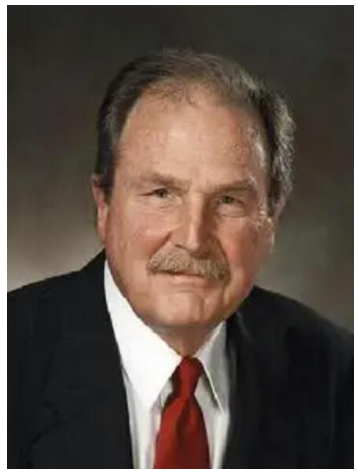
Optimality of Huffman Codes

- **Theorem 5.8.1.** Huffman coding is *optimal*, that is, if C^* is a Huffman code and C' is any other uniquely decodable code, $L(C^*) \leq L(C')$.

Remark. Huffman coding is a *greedy algorithm* in which it merges the two least likely symbols at each step.

LOCAL OPT $\rightarrow$ GLOBAL OPT

David Huffman



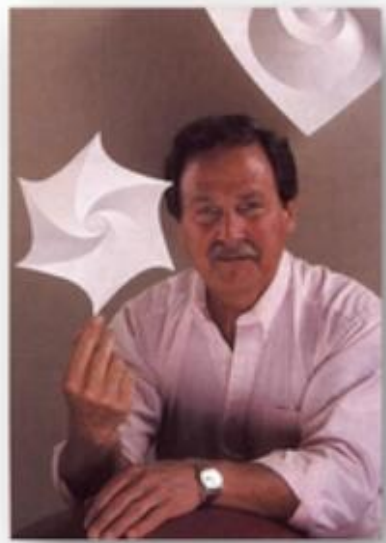
"Huffman code is one of the fundament ideas that people in computer science and date communications are using all the time."

Donald Knuth

August 9, 1925 - October 7, 1999

Huffman earned his bachelor's degree in electrical engineering from Ohio State University in 1944. Then, he served two years as an officer in the United States Navy. He returned to Ohio State to earn his master's degree in electrical engineering in 1949. In 1953, he earned his Doctor of Science in electrical engineering at the Massachusetts Institute of Technology (MIT), with the thesis The Synthesis of Sequential Switching Circuits, advised by Samuel H. Caldwell

David Huffman



David Albert Huffman
(1925 – 1999)

He then served in the U.S. Navy as a radar maintenance officer on a destroyer that helped to **clear mines in Japanese and Chinese waters after World War II**.

"Huffman code is one of the fundamental ideas that people in computer science and data communications are using all the time"
Donald E. Knuth, Stanford University

- Huffman worked on the problem for months, developing a number of approaches, but none that he could prove to be the most efficient. **Finally, he despaired of ever reaching a solution and decided to start studying for the final. Just as he was throwing his notes in the garbage, the solution came to him.** "It was the most singular moment of my life," Huffman says. "There was the absolute lightning of sudden realization."
- Huffman says he might never have tried his hand at the problem—much less solved it at the age of 25—if he had known that Fano, his professor, and Claude E. Shannon, the creator of information theory, had struggled with it. "It was my luck to be there at the right time and also **not have my professor discourage me by telling me that other good people had struggled with this problem,**" he says.
- https://www.huffmancoding.com/my-uncle/scientific-american

INFORMATION THEORY & CODING